

Testing the Student API

Test the API from a web page instead of typing `curl` commands.

Learning Goals

- Run the `run1-4-tests` page against the API and read its pass or fail results.
- Explain what each test checks: status codes, saved fields, and validation.
- Describe how a test is written (a request plus a list of checks) and why CORS lets the page call the API.

In `4-2`, you tested the API with `curl`. The same commands are in `test1-4.sh`.

- `curl` is fast for one request, but you must read every answer yourself.
- A **test page** sends many requests and tells you **pass or fail** for each check.
- The page is plain HTML and JavaScript. It lives in `run1-4-tests/`.

Test from the Browser: run1-4-tests

```
# 1. Start the server (from student-api/)  
php artisan serve --port=8080
```

```
# 2. Open the test page in a browser (from the lesson folder)  
open run1-4-tests/index.html          # macOS  
# WSL2: open the file from Windows Explorer
```

- The page calls the API with `fetch()` .
- It works with any server that has the same `/api` routes from `run1-4.sh` .
- Change the **API Base URL** at the top if your server uses another port.

What the Tests Check

Test	Request	What it checks
1.1 Connection	GET /api/students	200 , success , data is an array
1.2 API info	GET /api/test	message, endpoints, database is studentdb

Test	Request	What it checks
2.1 Get all	GET /api/students	total equals the array length
2.2 Get one	GET /api/students/{id}	correct student, unknown id gives 404
2.3 Create	POST /api/students	201 , saved fields match
2.4 Update	PUT /api/students/{id}	one field changes, others stay
2.5 Delete	DELETE /api/students/{id}	200 , then 404

Test	Request	What it checks
3.1 Validation	POST with bad data	422 and an errors object
3.2 Search, stats	GET .../search , .../stats	results and a 400 without a name

- **Run All Tests** runs them in order. Test data is created and then deleted.

How One Test Works

Every test sends a request and lists **checks**: a description and true or false.

```
async function testGetOne() {  
  const r = await request('/students/1');           // GET /api/students/1  
  const missing = await request('/students/999999');  
  return show('t22', [  
    ['status is 200', r.status === 200],  
    ['id matches', r.data.data.id === 1],  
    ['unknown id returns 404', missing.status === 404]  
  ], r.data);  
}
```

- The test passes (green) only when **every** check is true. A failed check is listed in red.
- `request()` adds the header `Accept: application/json`, so Laravel answers `404` and `422` with JSON.
- This is the same idea as the `curl` commands in `test1-4.sh`, written in JavaScript.

Why Can a Web Page Call the API?

A page opened from a file (or another port) calls `http://localhost:8080`. Browsers allow this only if the server agrees: this is **CORS**.

- Laravel answers `/api/*` with `Access-Control-Allow-Origin: *`. You saw this header in the `curl -i` output earlier.
- If a test shows *no response from the server*, check that `php artisan serve` is running and that the API Base URL is correct.
- The test page has no login. In the next lesson, `5. Bearer Token APIs/run1-5-tests` adds a token to every write request.